

SINCRONIZACIÓN EN SISTEMAS OPERATIVOS

La **sincronización** es el conjunto de mecanismos que permiten coordinar la ejecución concurrente de procesos o hilos, garantizando el **acceso seguro a recursos compartidos** y el **orden correcto de ejecución**.

♦ Objetivos principales

1. **Exclusión mutua:** asegura que solo un proceso a la vez acceda a una sección crítica (un recurso compartido).
2. **Ordenación y coordinación:** algunos procesos deben esperar que otros completen ciertas tareas antes de continuar.
3. **Evitar condiciones de carrera:** donde el resultado depende del orden de ejecución de los procesos.

♦ Mecanismos de sincronización comunes

Stallings clasifica varios mecanismos de concurrencia:

Mecanismo	Descripción
Semáforo	Entero usado para señalización; operaciones atómicas wait (P) y signal (V). Puede bloquear o desbloquear procesos.
Mutex	Versión binaria del semáforo; solo el proceso que lo bloquea puede liberarlo.
Variable de condición	Permite que un hilo espere hasta que se cumpla cierta condición dentro de un monitor.
Monitor	Estructura de software que encapsula datos y procedimientos, garantizando exclusión mutua automáticamente.
Mensajes o Mailboxes	Comunicación por paso de mensajes, también sirve para sincronizar.
Spinlocks / Busy waiting	Espera activa en un bucle hasta que se libera el recurso.

MONITORES

El **monitor** es un **mecanismo de sincronización de alto nivel** propuesto por C. A. R. Hoare y refinado por Lampson y Redell.
Se implementa normalmente dentro de los lenguajes de programación concurrente.

♦ Definición

Un **monitor** es un módulo de software que agrupa **variables compartidas, procedimientos y código de inicialización**, garantizando que **solo un proceso a la vez** pueda ejecutar alguno de sus procedimientos.

♦ Características fundamentales

1. Las **variables locales** del monitor son accesibles **solo** por los procedimientos del monitor.
2. **Los procesos entran al monitor invocando sus procedimientos.**
3. **Solo un proceso puede estar ejecutando dentro del monitor a la vez** (el resto queda bloqueado en una cola de entrada).

Esto provee **exclusión mutua automática**, sin necesidad de semáforos explícitos dentro del código del usuario.

SINCRONIZACIÓN DENTRO DEL MONITOR

Los monitores usan **variables de condición**, que son una especie de “puntos de espera” para los procesos.

Operaciones básicas sobre variables de condición:

- **cwait(c)** → Suspende el proceso que la llama en la cola asociada a la condición **c**, liberando el monitor para otros procesos.
- **csignal(c)** → Despierta (reanuda) un proceso que estaba esperando en la condición **c**.

Si no hay procesos esperando en **c**, el **csignal** se pierde (no se acumula).



EJEMPLO CLÁSICO: Productor–Consumidor con monitor

En Stallings (figura 5.19), se resuelve el problema del **buffer limitado** así:

```
monitor boundedbuffer;
char buffer[N];
int nextin, nextout, count;
cond notfull, notempty;

void append(char x) {
    if (count == N) cwait(notfull);    // Espera si el buffer está
    lleno
    buffer[nextin] = x;
    nextin = (nextin + 1) % N;
    count++;
    csignal(notempty);                // Notifica que hay datos
}

void take(char x) {
    if (count == 0) cwait(notempty);   // Espera si el buffer está
    vacío
    x = buffer[nextout];
    nextout = (nextout + 1) % N;
    count--;
    csignal(notfull);                 // Notifica que hay espacio
}
```

- ♦ **Exclusión mutua:** el monitor garantiza que solo un proceso a la vez acceda al buffer.
 - ♦ **Sincronización:** las variables `notfull` y `notempty` regulan cuándo los productores o consumidores deben esperar o continuar.
-

MODELOS DE MONITORES

Stallings distingue dos implementaciones:

1. Monitor de Hoare

- El proceso que realiza `csignal` **cede inmediatamente** el monitor al proceso despertado.
- Preciso, pero menos modular (el código de señalización debe conocer las condiciones específicas del otro proceso).

2. Monitor de Lampson/Redell

- `csignal` solo “avisa”; el proceso despertado **debe verificar por sí mismo** si la condición aún se cumple.
- Más modular, evita errores de “despertar erróneo” y mejora la abstracción.



DIFERENCIAS MONITOR vs SEMÁFORO

Aspecto	Semáforo	Monitor
Nivel	Bajo nivel (mecánico)	Alto nivel (estructurado)
Exclusión mutua	Debe implementarse manualmente	Automática
Sincronización	Control mediante P/V (wait/signal)	Mediante variables de condición
Errores comunes	Olvidar liberar semáforo o hacerlo de más	Menos propenso: toda la lógica está encapsulada
Modularidad	Global y propenso a errores	Local, autocontenida



VENTAJAS DE LOS MONITORES

- Fomentan la **modularidad** y **abstracción de datos**.
- Reducen errores en sincronización compleja.
- Son base de mecanismos modernos como **monitores de Java** (`synchronized`, `wait()`, `notify()`).
- Facilitan la verificación formal del código concurrente.

1 Diferencia base entre procesos e hilos

Aspecto	Proceso	Hilo (thread)
Unidad básica de ejecución	Programa independiente	Subunidad de un proceso
Espacio de memoria	Tiene su propio espacio de direcciones	Comparte el espacio de memoria del proceso padre
Comunicación	Requiere mecanismos IPC (pipes, sockets, colas de mensajes, etc.)	Comunicación directa a través de variables compartidas
Creación	Costosa (crear PCB, asignar memoria, etc.)	Rápida (usa recursos del proceso padre)
Fallos	Si un proceso falla, no afecta a los demás	Si un hilo falla, puede corromper todo el proceso

2 SINCRONIZACIÓN ENTRE PROCESOS

✓ Ventajas

- 🧱 **Aislamiento total:** cada proceso tiene su memoria; un error en uno no daña a los demás.
- 🛡️ **Seguridad y estabilidad:** útil en sistemas donde se requiere protección (por ejemplo, navegador con procesos por pestaña).
- 🏢 **Distribución sencilla:** procesos pueden ejecutarse incluso en distintas máquinas (modelo cliente-servidor o microkernel).

✗ Desventajas

- 🐢 **Comunicación más lenta:** requiere **mecanismos de IPC** (Inter-Process Communication) como *pipes*, *sockets*, *message queues*.
- ⚙️ **Mayor sobrecarga del sistema operativo:** crear, destruir y cambiar de contexto entre procesos implica más tiempo.
- 🏠 **Sincronización más compleja:** se necesitan estructuras externas (semáforos de sistema, memoria compartida protegida, etc.).

📘 **Ejemplo típico:** procesos sincronizados por semáforos del sistema para controlar acceso a un archivo compartido.

3 SINCRONIZACIÓN ENTRE HILOS

Ventajas

-  **Comunicación rápida:** los hilos comparten el mismo espacio de memoria y variables globales.
-  **Uso eficiente de recursos:** varios hilos dentro del mismo proceso comparten código y memoria, reduciendo duplicación.
-  **Bajo costo de creación y cambio de contexto:** mucho más rápido que crear procesos separados.
-  **Ideal para paralelismo fino:** dividir una tarea grande (por ejemplo, renderizar una imagen o manejar múltiples clientes en un servidor).

Desventajas

-  **Menor aislamiento:** un error en un hilo (por ejemplo, escritura fuera de rango) puede afectar a todo el proceso.
-  **Problemas de sincronización:** al compartir memoria, hay riesgo de *race conditions* y *deadlocks* si no se usan mutex, monitores o semáforos correctamente.
-  **Depuración más difícil:** los errores de concurrencia son difíciles de reproducir.

 **Ejemplo típico:** un servidor web donde cada hilo atiende a un cliente, usando monitores o mutex para acceder al log compartido.